

Task 1 - Базова архітектура мікросервісів (10 балів)

Це завдання є основою для наступних завдань, у яких будуть додаватись нові програмні засоби та функціональність.

У завданні необхідно реалізувати частину банківської системи на основі трьох мікросервісів, взаємодія між якими відбувається на основі HTTP-протоколу, бажано на основі парадигми REST API (можливо також використання gRPC-протоколу).

Приклад реалізації можна подивитись на відео: https://youtu.be/w5EBi2nT_B0

Архітектура складається з трьох мікросервісів:

- **facade-service** - приймає POST/GET платіжні транзакції від клієнтів
- **logging-service** - зберігає у пам'яті всі повідомлення/транзакції які йому надходять, та може повертати їх
- **counter-service** - для кожного клієнта підраховує остаточний баланс на його рахунку, застосовуючи операції "додавання"/"віднімання" сум вказаних у транзакціях. При зверненні до сервісу повертає баланс на рахунок відповідного клієнта, або на всіх рахунках
- кожен сервіс має окремо запускатись у Docker-контейнері, а загальне "керування" контейнерами має здійснюватись через Docker Compose

Базова функціональність системи

Клієнт взаємодіє з **facade-service** через HTTP POST та GET запити. У якості клієнта може бути *curl*, *Postman*, браузер у Dev mode та інше

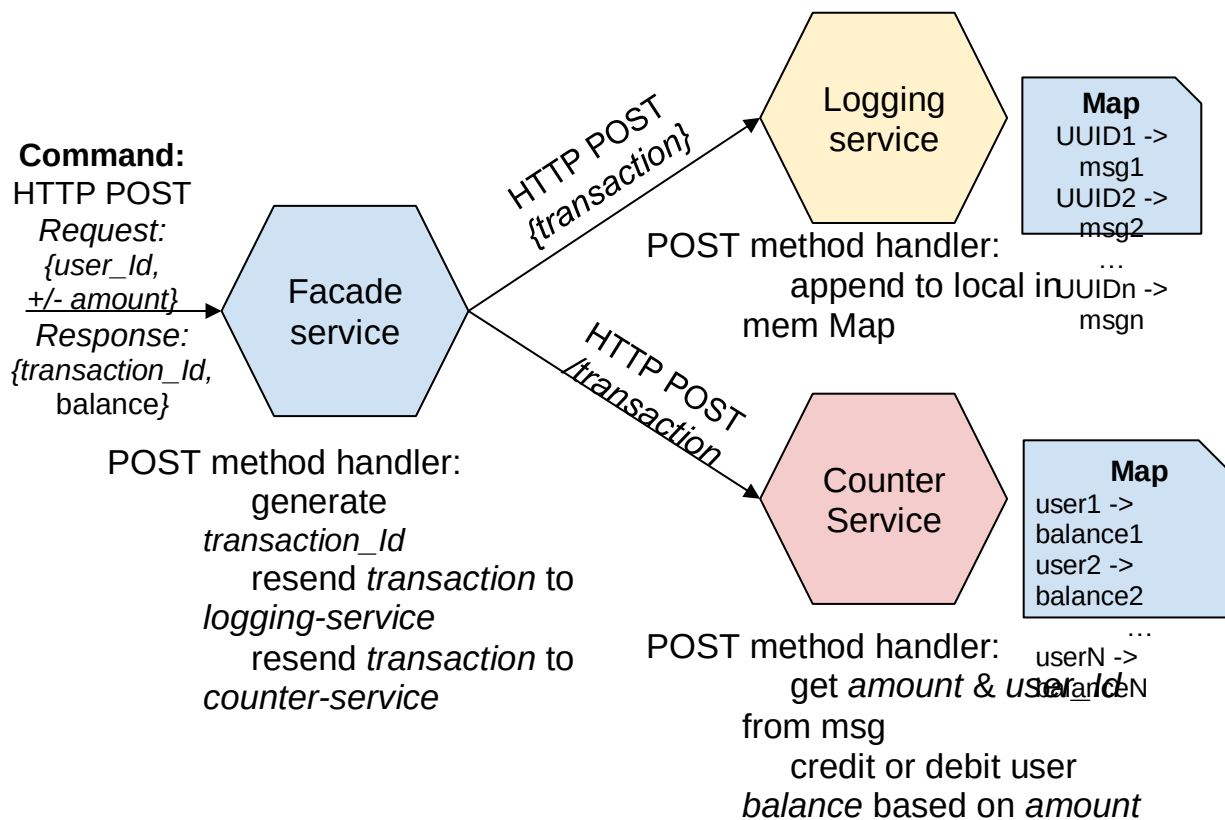
- **Формат повідомлень/транзакції клієнта**

- *{user_Id, +/- amount}*

- **Опис HTTP POST request flow**

- Клієнт відправляє POST запит на **facade-service**, з транзакцією, в якій міститься сума на яку треба поповнити чи дебетувати (зняти) рахунок та ідентифікатор клієнта - *msg*
- Отримавши повідомлення **facade-service** генерує для нього *transaction_ID* та *timestamp* (це насправді може бути одне значення - timestamp as unique ID)

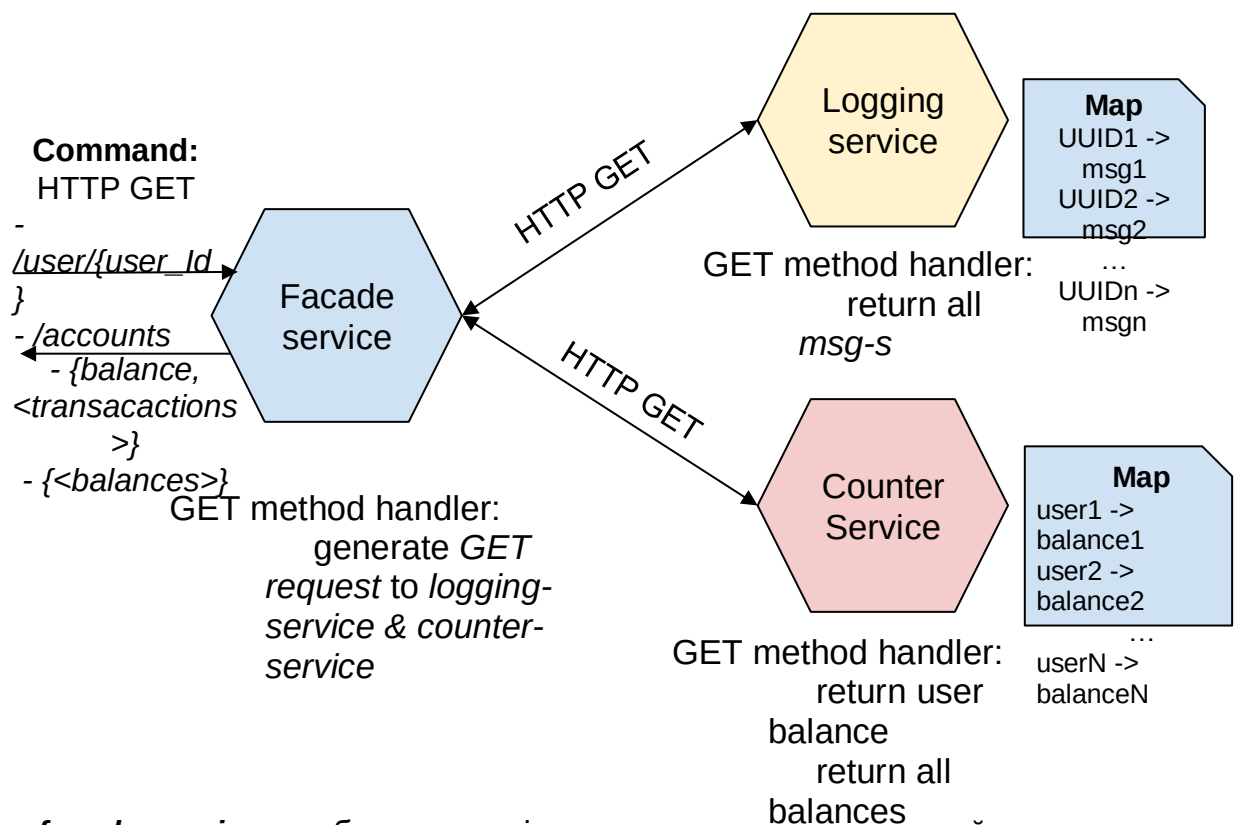
- Повідомлення $\{transaction_ID, user_Id, +/- amount\}$ за допомогою програмного REST/HTTP-client у вигляді POST-повідомлення пересилається на **logging-service** де зберігаються в оперативній пам'яті
- одночасно, повідомлення пересилаються на **counter-service**, на якому виконується операція з поповнення чи дебетування рахунку клієнта, і повертає **facade-service** оновлене значення балансу
- **logging-service** отримавши повідомлення зберігає його у локальну хеш-таблицю ($transaction_ID$ - у якості ключа) (для відлагодження може виводити у свою консоль отримане повідомлення)
- **counter-service** зберігає баланси рахунків клієнтів у пам'яті у вигляді хеш-таблиці, з ключем $user_Id$ і балансом рахунку у якості значення
- **facade-service** чекає відповідей від **logging-service** та **counter-service**, і після цього надсилає клієнту відповідь, яка містить $\{transaction_ID, balance\}$



● Опис HTTP GET request flow

- **facade-service** підтримує два види GET запитів:

- *GET http://facade-service/user/{user_id}* - повертає значення балансу у вказаного клієнта (бере з *counter-service*) та список всіх його транзакцій (бере з *logging-service*) - *{balance, <transacactions>}*
- *GET http://facade-service/accounts* - повертає значення балансу всіх клієнтів (бере з *counter-service*) - *{<balances>}*
- Отримавши запит **facade-service** генерує GET-запити до **logging-service** та **counter-service** за допомогою програмного REST/HTTP-client, та на основі їх відповідей формує відповідь клієнту



- на **facade-service** має бути можливість визначати скільки часу займає запит окремо до **logging-service** та **counter-service** - це знадобиться потім для порівняння швидкості в залежності від імплементації цих сервісів
- як варіант реалізації, рекомендується просто виміряти час віддаленого виклику до сервісів, та накопичувати його у двох змінних
- для зчитування часу чи скидання попередніх вимірювань може бути додані додаткові HTTP методи до **facade-service**

Тестування продуктивності

Необхідно створити (HTTP-) клієнта, який може робити задану кількість викликів до **facade-service**, а також заміряти загальний час витрачений на здійснення цих викликів.

У якості клієнта може бути утиліта *curl* (чи будь-яка інша утиліта командного рядка), скрипт на Python (чи будь-якій іншій мові програмування)

В залежності від одночасної кількості клієнтів треба визначати яку кількість запитів в секунду обробляє система.

Це обраховується як - *count/time*

time - кількість часу (секунд) яка була сумарна витрачена на здійснення всіх запитів

За допомогою розробленого клієнта необхідно протестувати продуктивність та коректність нашої системи у двох сценаріях:

1. 10 клієнтів одночасно роблять по 10K однакових транзакцій по додаванню 1 на свій рахунок. У результаті кінцеве значення балансу на 10-и рахунках має бути по 10K.
2. 10 клієнтів одночасно роблять по 10K однакових транзакцій по додаванню 1 на один і той самий рахунок. У результаті кінцеве значення балансу на одному рахунках має бути 100K.

Під час тестів треба визначити:

- загальний час та кількість запитів в секунду
- якій внесок в цей загальний час вносять виклики до **logging-service** та **counter-service**

Протокол

1. Має бути продемонстрована коректність та функціональність системи для простих випадків для декількох запитів
 - a. Зробити декілька транзакцій для різних клієнтів (з +/- *amount*)
 - b. Через GET запити показати коректність результатів
2. Провести тестування продуктивності для двох наведених сценаріїв
3. Проаналізувати отримані результати щодо продуктивності

Вимоги до реалізації

Мікросервіси представляють собою три окремі Веб-застосування, які можуть запускатись незалежно одне від одного.

Мова програмування та фреймворки можуть бути будь-які (але без зайвого ускладнення).

Код всіх три мікросервісів має бути збережений у GitHub в одному репозиторії у гілці *micro basics*

Вимоги до протоколу

У протоколі має міститись:

- посилання на GitHub з проектом
- скріншоти з POST/GET-запитами з додаванням декількох повідомлень та відповіді на них
- вміст консолі кожного з мікросервісу якщо туди виводиться якась корисна інформація